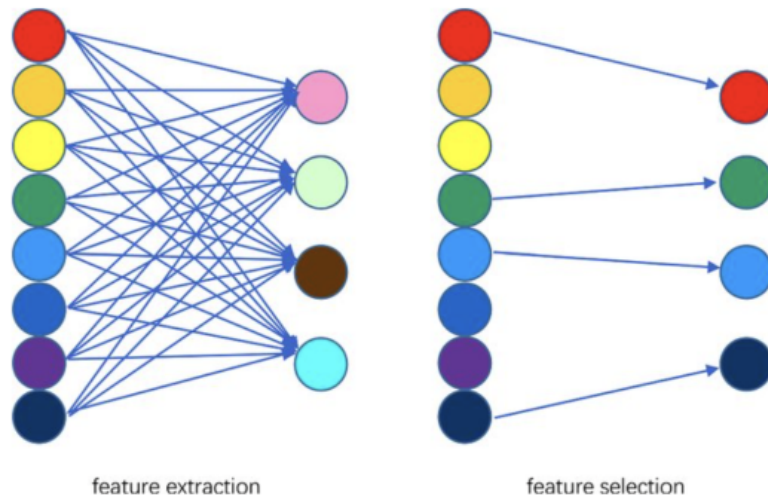


session 6

Feature Selection: Select a subset from a given feature set.

Feature Extraction: A linear or non-linear transform from the original feature space to a lower dimension space.

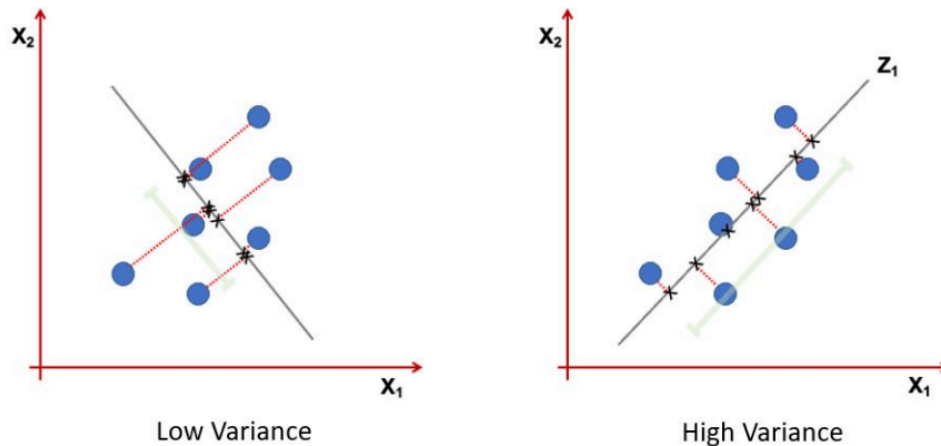


Maximize retention of **important information** while reducing dimensionality.

What is **important information**?

Variance of Data

- Maximize retention of important information while reducing dimensionality.
- Information: Variance of projected data



▼ We want to find **the most important directions** in our data.

These are directions where the data **varies the most** (has the most spread). If we find these directions, we can project the data onto them and reduce dimensions **while keeping as much information as possible**.

So,

👉 **How do we mathematically find the directions with the most variance?**

Center the Data (Subtract the Mean)

- Mean-center the data: Zeroing out the mean of each feature.
- Scaling to normalize each feature to have variance 1 (an arbitrary step).
Might affect results. It helps when unit of measurements of features are different and some features may be ignored without normalization.

Compute the Covariance Matrix

Covariance is a measure of how much two random features vary together (covariance is symmetric)

$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])] = E[(Y - E[Y])(X - E[X])] = \text{Cov}(Y, X)$$

A covariance matrix generalizes the concept of covariance to multiple features

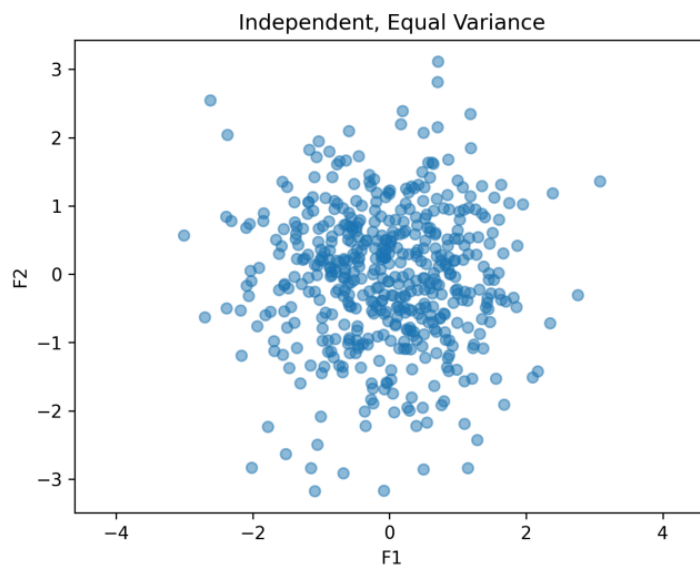
For a random vector

$$F = [F1, F2, \dots, Fd] :$$

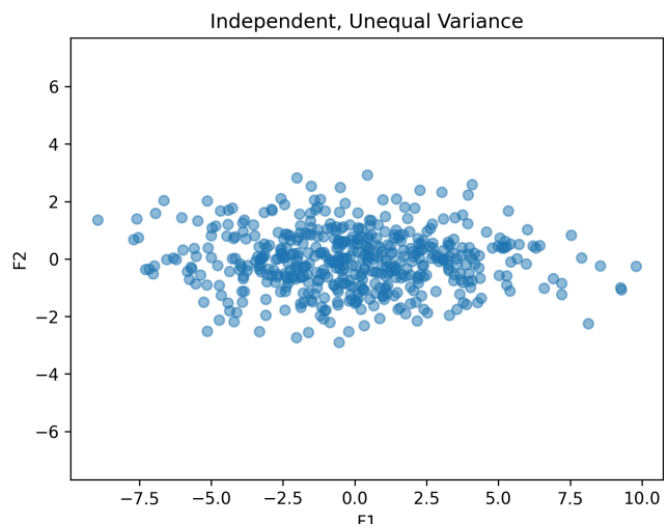
$$\Sigma = \begin{pmatrix} \text{Var}(F_1) & \text{Cov}(F_1, F_2) & \cdots & \text{Cov}(F_1, F_d) \\ \text{Cov}(F_2, F_1) & \text{Var}(F_2) & \cdots & \text{Cov}(F_2, F_d) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(F_d, F_1) & \text{Cov}(F_d, F_2) & \cdots & \text{Var}(F_d) \end{pmatrix}$$

The diagonal elements are the variances, and off-diagonal elements are covariances.

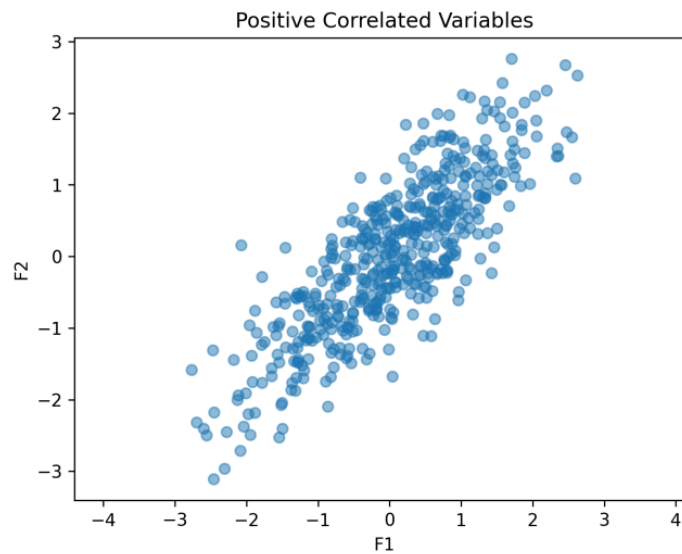
If $\Sigma = \begin{pmatrix} a & 0 \\ 0 & a \end{pmatrix}$, then:



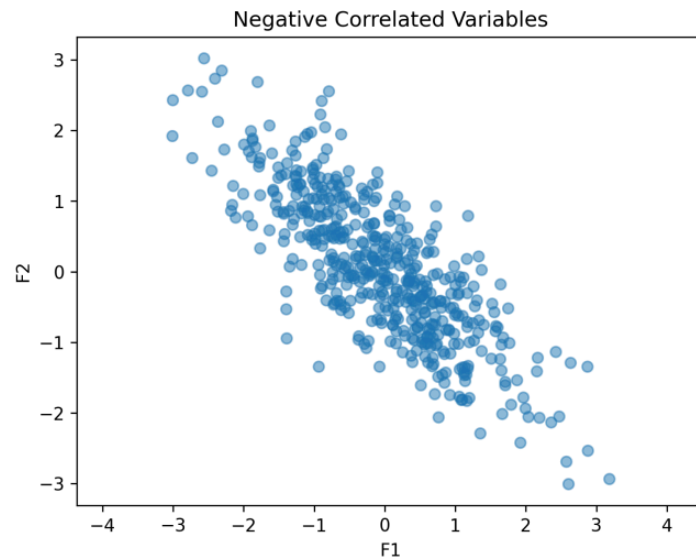
If $\Sigma = \begin{pmatrix} a & 0 \\ 0 & d \end{pmatrix}$ and $a > d$, then:



If $\Sigma = \begin{pmatrix} a & b \\ b & d \end{pmatrix}$, $a > d$, and $b > 0$, then:



If $\Sigma = \begin{pmatrix} a & b \\ b & d \end{pmatrix}$, $a > d$, and $b < 0$, then:



If our dataset X' (after centering) has **rows = data points** and **columns = features**, then:

$$\Sigma = \frac{1}{N-1} X'^T X' = C$$

???? find the directions with the most variance

$$\max_v v^T C v \quad ; \quad v^T v = 1$$

1. v is a **direction vector** (also called an eigenvector).
2. C is the **covariance matrix** of the data. It captures how features (variables) in the dataset relate to each other.
3. $v^T C v$ calculates the variance along the direction v . a **scalar value** that represents the spread (variance) of the data along v .

Compute Eigenvalues and Eigenvectors

To solve this, we use the
Lagrange multiplier method:

$$L(v, \lambda) = v^T C v - \lambda(v^T v - 1)$$

This simplifies to: $Cv = \lambda v$

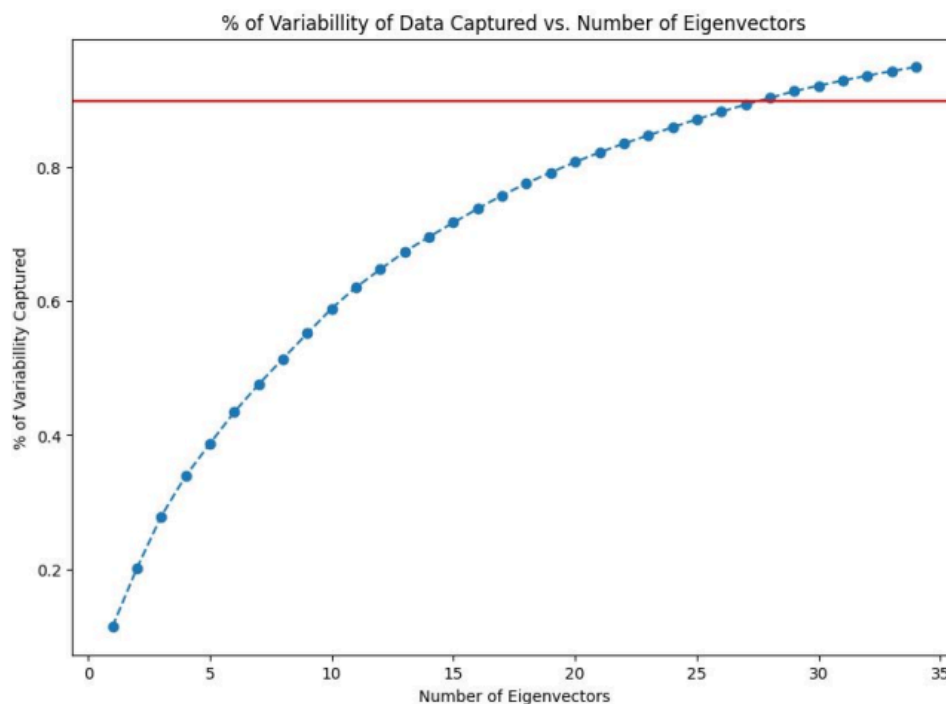
$$(C - \lambda I)v = 0$$

$$\det(C - \lambda I) = 0$$

We find all $(v_1, \lambda_1), (v_2, \lambda_2), \dots, (v_k, \lambda_k)$ as the k eigenvectors of C having largest eigenvalues: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_k$

Select the desired variance ratio and select the principal components.(Explained Variance)

$$\min_k \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^d \lambda_i} \geq 0.9$$



Loadings indicate how much each original feature contributes to a principal component (PC). They help us understand the **meaning** of each PC in terms of the original variables.

Mathematically, **loadings** are the **coefficients** (weights) of the original features in the principal component space.

For a given **eigenvector** v_k corresponding to the k^{th} principal component, the **loading matrix** is:

$$L = V \cdot \sqrt{\Lambda}$$

- V is the matrix of **eigenvectors** (principal axes).

- Λ is the **diagonal matrix of eigenvalues** (variance explained by each PC).
- L contains **loadings**, showing how much each feature contributes to each principal component.

Interpreting Principal Components: Loadings tell us **which features influence each principal component the most**.

Feature Importance in Dimensionality Reduction: If a feature has **low loadings for all PCs**, it contributes little and may be removed for simpler models.

Example

1. **Prefrontal Cortex (PFC)**
2. **Hippocampus (HPC)**
3. **Amygdala (AMY)**
4. **Visual Cortex (VC)**
5. **Motor Cortex (MC)**

Subject	PFC-HPC	PFC-AMY	PFC-VC	PFC-MC	HPC-AMY	...
1	0.45	0.62	0.30	0.50	0.40	...
2	0.50	0.55	0.28	0.48	0.42	...
...	...	...	...	...	...	...
100	0.41	0.60	0.32	0.51	0.38	...

Feature (Brain Connectivity)	PC1 (Default Mode)	PC2 (Emotion)	PC3 (Sensory-Motor)
PFC-HPC (Prefrontal ↔ Hippocampus)	0.75	0.10	0.05
PFC-AMY (Prefrontal ↔ Amygdala)	0.72	0.80	0.10
PFC-VC (Prefrontal ↔ Visual Cortex)	0.60	0.20	0.85
PFC-MC (Prefrontal ↔ Motor Cortex)	0.55	0.15	0.80
HPC-AMY (Hippocampus ↔ Amygdala)	0.50	0.88	0.12

- **PC1 (Default Mode Network - DMN)**
 - Strong loadings on **PFC-HPC (0.75)** and **PFC-AMY (0.72)** → Suggests that this PC represents the **Default Mode Network**, known for memory and self-referential thinking.
- **PC2 (Emotion Regulation Component)**
 - High contribution from **PFC-AMY (0.80)** and **HPC-AMY (0.88)** → Suggests an emotion-related network (PFC-Amygdala interactions are crucial in emotion regulation).
- **PC3 (Sensory-Motor Integration Component)**
 - High contributions from **PFC-VC (0.85)** and **PFC-MC (0.80)** → Suggests this PC reflects **sensory-motor control**, linking the visual and motor cortex.

Overfitting happens when a model

memorizes noise instead of learning general patterns. Large weights in a model can **exaggerate small variations** in the training data, making the model **too sensitive** to the specific dataset.

Large Weights Lead to High Model Sensitivity

$$h_w(x) = w_0 + w_1x_1 + w_2x_2 + \dots + w_Dx_D + b = y$$

If w_i values are **too large**, even a **tiny change in x_i** can cause a **huge change in y** . This means:

The model **reacts too much** to minor variations in data.

It **fits noise instead of actual trends**.

In regression, we minimize the Mean Squared Error (MSE):

$$J(w) = \sum_{i=1}^N (y^{(i)} - w^T x^{(i)})^2$$

$$MSE = \sum_{i=1}^N (y^{(i)} - y'^{(i)})^2$$

If weights w are large:

- The model fits the training data **very well**, meaning **low training error**.
- However, on new (test) data, predictions **become unstable**.
- This results in **poor generalization** (high test error).

$$J_\lambda(w) = \underbrace{J(w)} + \underbrace{\lambda R(w)}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data

A **penalty term** is an additional component added to the loss function to **control the complexity** of the model. It prevents overfitting by discouraging large model parameters (e.g., large weights in regression or SVM). This technique is known as **regularization**.

✓ L1 Regularization

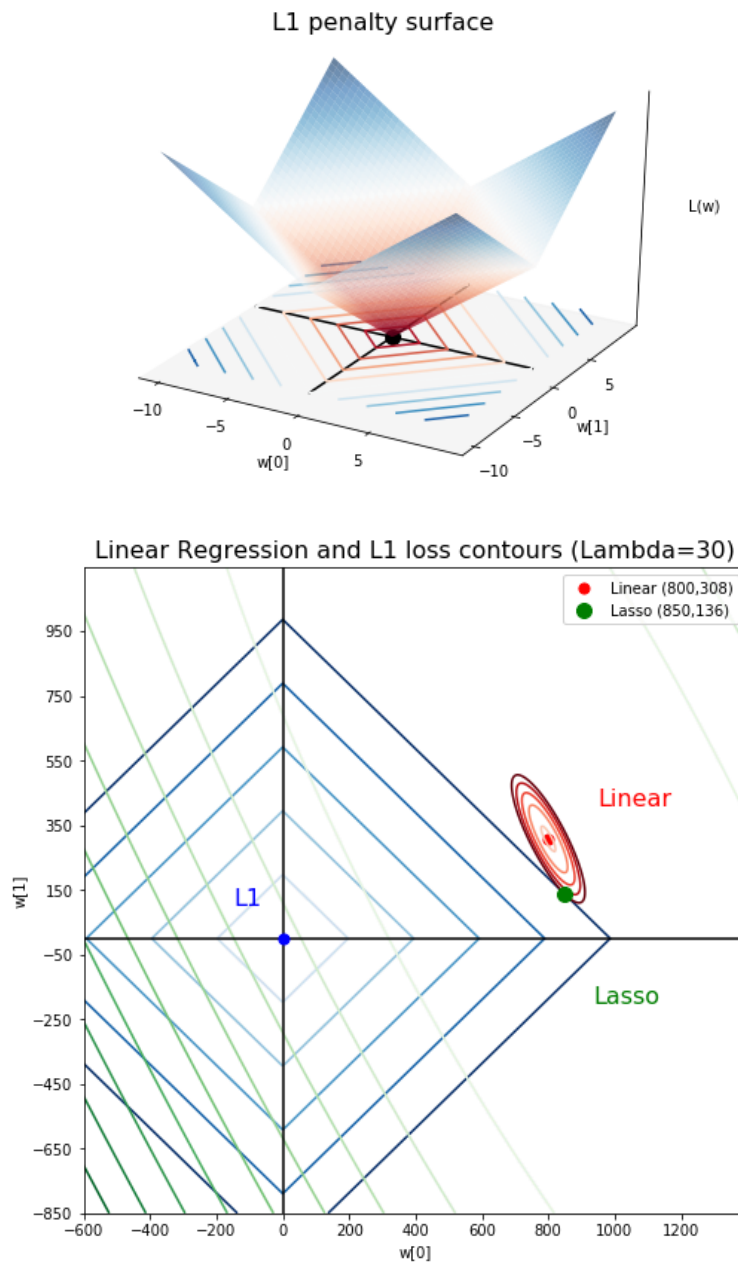
Penalty Term: Sum of Absolute Weights: $\lambda \sum |w|$

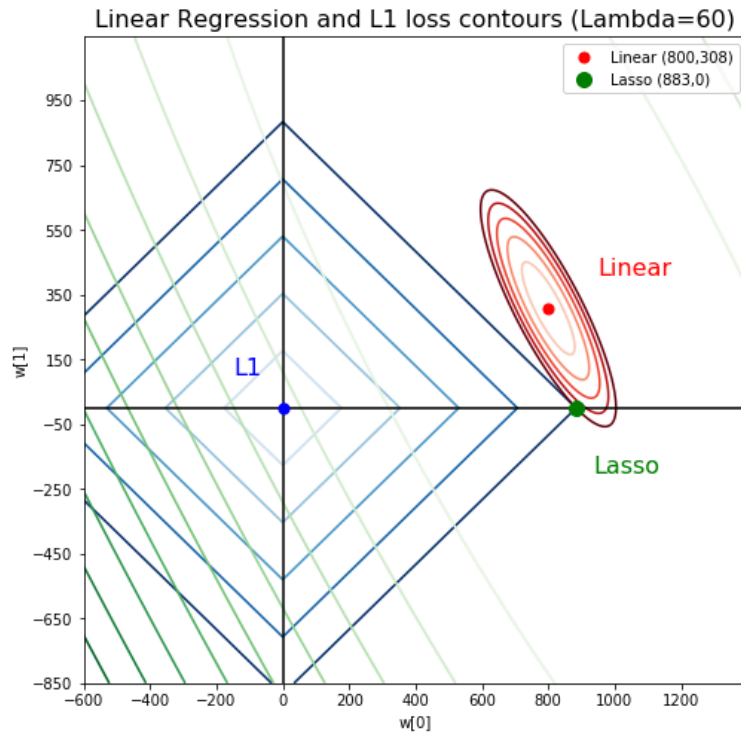
$$Loss = \sum_{i=1}^N (y^{(i)} - y'^{(i)})^2 + \lambda \sum |w|$$

Lasso Regression: Shrinks some weights **to zero**, performing **feature selection**

Large λ → Stronger regularization, more coefficients set to zero.

Small λ → Less regularization, more nonzero weights.



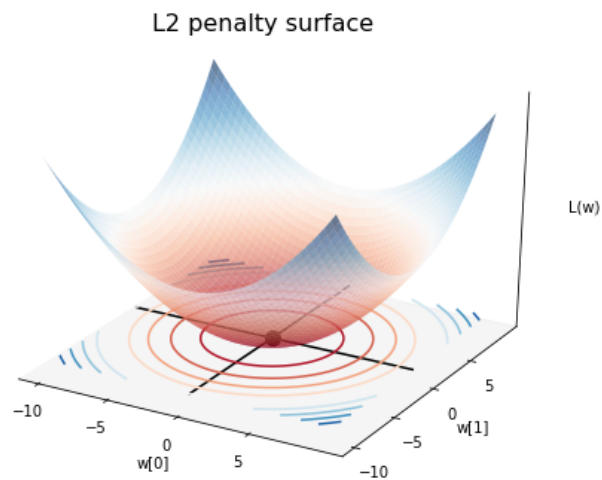


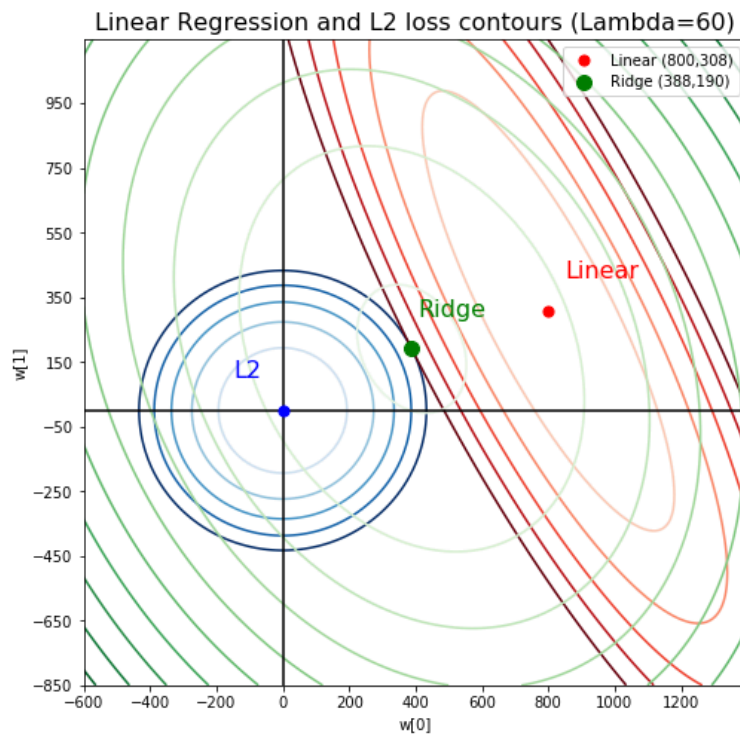
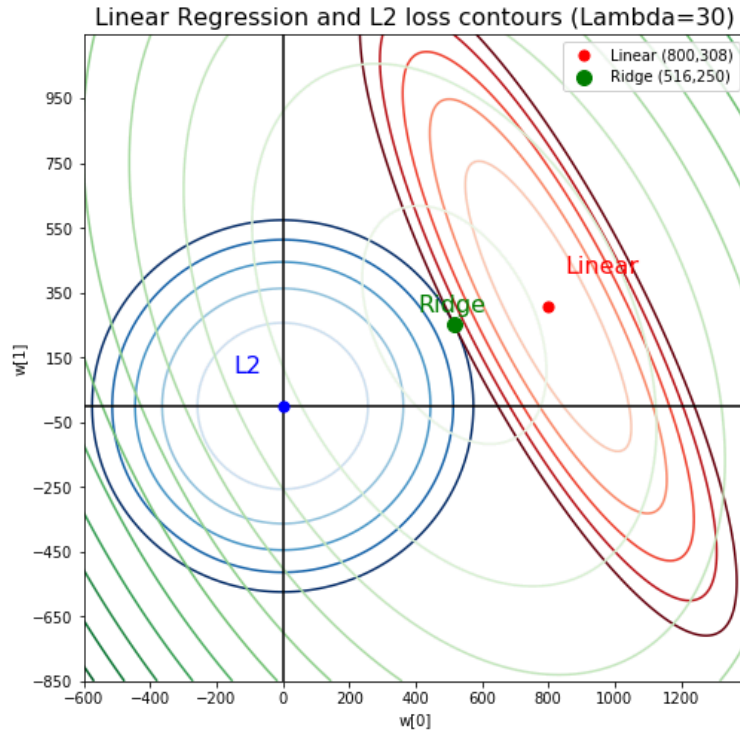
✓ L2 Regularization

Penalty Term: Sum of Squared Weights: $\lambda \sum w^2$

$$Loss = \sum_{i=1}^N (y^{(i)} - y'^{(i)})^2 + \lambda \sum w^2$$

Ridge Regression: Encourages smaller weights, reducing overfitting but not removing features.

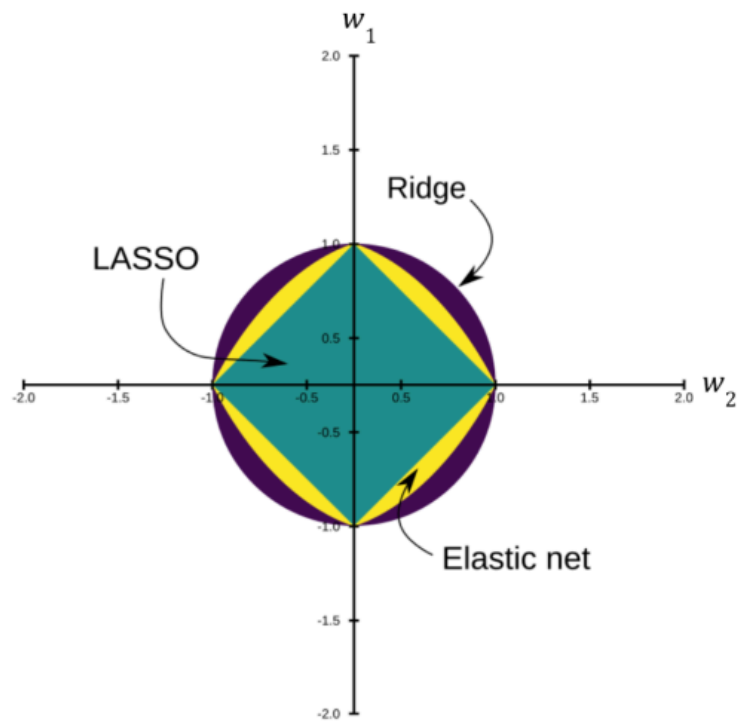




✓ Elastic Net (Combination of L1 & L2)

$$Loss = \sum_{i=1}^N (y^{(i)} - y'^{(i)})^2 + \lambda_2 \sum w^2 + \lambda_1 \sum |w|$$

Combines **Lasso (feature selection)** and **Ridge (shrinkage)**. for: High-dimensional datasets with **correlated features**.



Penalty contour for different norms

